Parallel and Distributed Systems

Chapter 2: Theoretical Aspects of Parallelization

Prof. Dr. Martin Sträßer

Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Overview

- Performance Metrics for Parallel Programs
- How to Make a Program Parallel?

Performance Metrics for Parallel Programs

Runtime of Parallel Programs

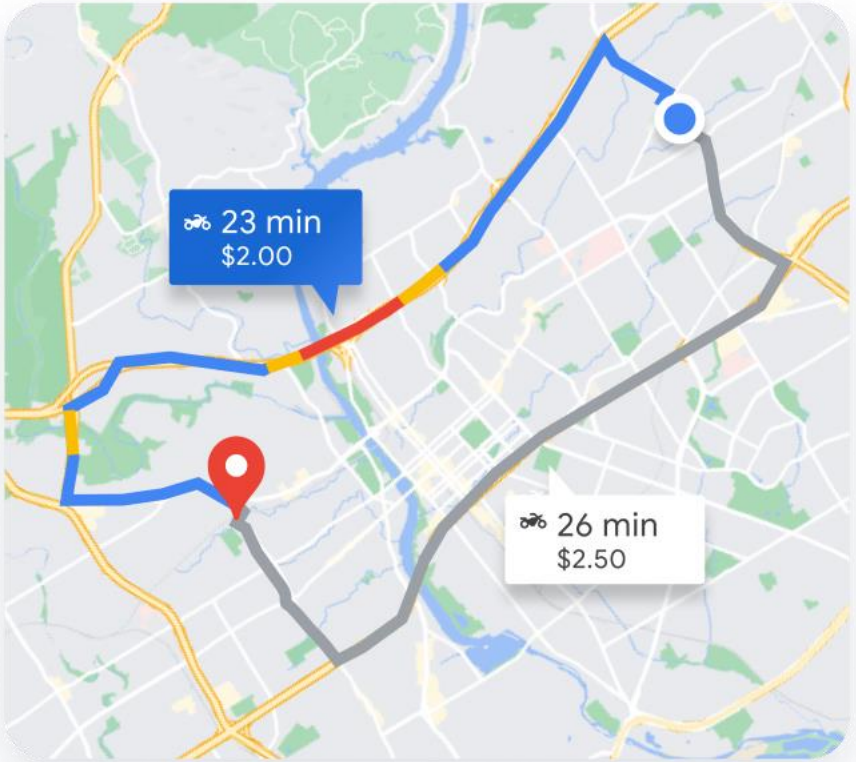
- Why to make programs parallel?
  - To reduce their runtime
- We denote the runtime of a parallel program as:

$$T_p(n)$$

- p is the number of processors/cores we use
- n is the *problem size*

Problem Size

- Problem size is a theoretical term and considers the fact that the operations needed to solve a problem and hence the runtime of a program depends on the input
- Examples
  - Matrix multiplication: It makes a difference whether we multiply 2x2- or 10000x10000-matrices
  - Route planning: It makes a difference whether we want to find the shortest path from Campus 1 to Campus 2 or from Berlin to Madrid



Runtime Components

- When we execute a program on multiple processors:
  - Tasks (threads/processes) have to be placed on a processor
  - The tasks have to a be started on the processors
  - The processors compute the tasks
  - Data may be exchanged between the tasks
  - Tasks may wait for other tasks to be completed before they can be started (e.g., when Task B requires (intermediate) results of Task A as its input)
  - Running tasks may need to be synchronized (e.g., when accessing/modifying shared resources)

Computing time	Communi-cation time	Waiting time	Synchroni-zation time	Placement time	Start time

$$T_p(n) = T_{\text{CPU}} + T_{\text{COM}} + T_{\text{WAIT}} + T_{\text{SYN}} + T_{\text{Place}} + T_{\text{Start}}$$

Discussion

- Which of these components could we as software engineers influence?

Computing time | Communi-cation time | Waiting time | Synchroni-zation time | Placement time | Start time

$$T_p(n) = T_{\text{CPU}} + T_{\text{COM}} + T_{\text{WAIT}} + T_{\text{SYN}} + T_{\text{Place}} + T_{\text{Start}}$$

Parallelization Overhead

- The parallelization overhead T_{CWS} consists of:
  - Communication time
  - Waiting time
  - Synchronization time

$$T_{\mathrm{CWS}} = T_{\mathrm{COM}} + T_{\mathrm{WAIT}} + T_{\mathrm{SYN}}$$

- The benefit of making a program parallel maximize if:
  - No data has to be exchanged between tasks
  - No task requires results from an other task
  - The tasks do not access/write to shared resources

Runtime Components

- The setup time T_{Setup} is the sum of the placement time and start time
- It depends on the hardware and operating system
- Typically, they cannot be controlled by software engineers

$$T_{\text{Setup}} = T_{\text{Place}} + T_{\text{Start}}$$

- The computing time T_{CPU} depends on the number of required CPU operations (e.g., additions, multiplications)
- Compilers and caches typically apply optimizations to minimize the number of CPU operations

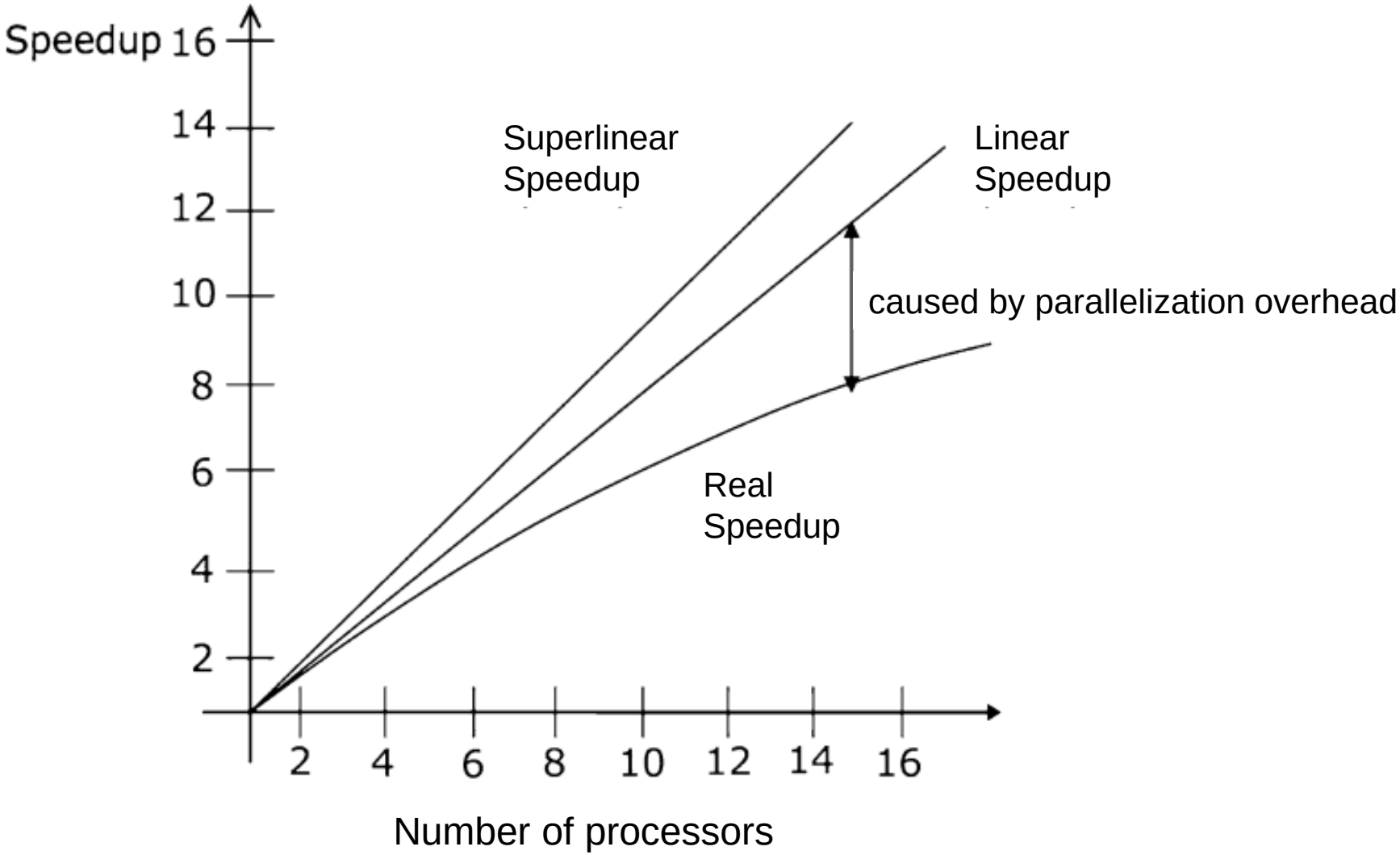
Speedup

- Let $T'(n)$ the runtime of the best sequential (non-parallel) problem solution / program implementation
- The speedup $S_p(n)$ is defined as the following ratio:

$$S_p(n) = \frac{T'(n)}{T_p(n)}$$

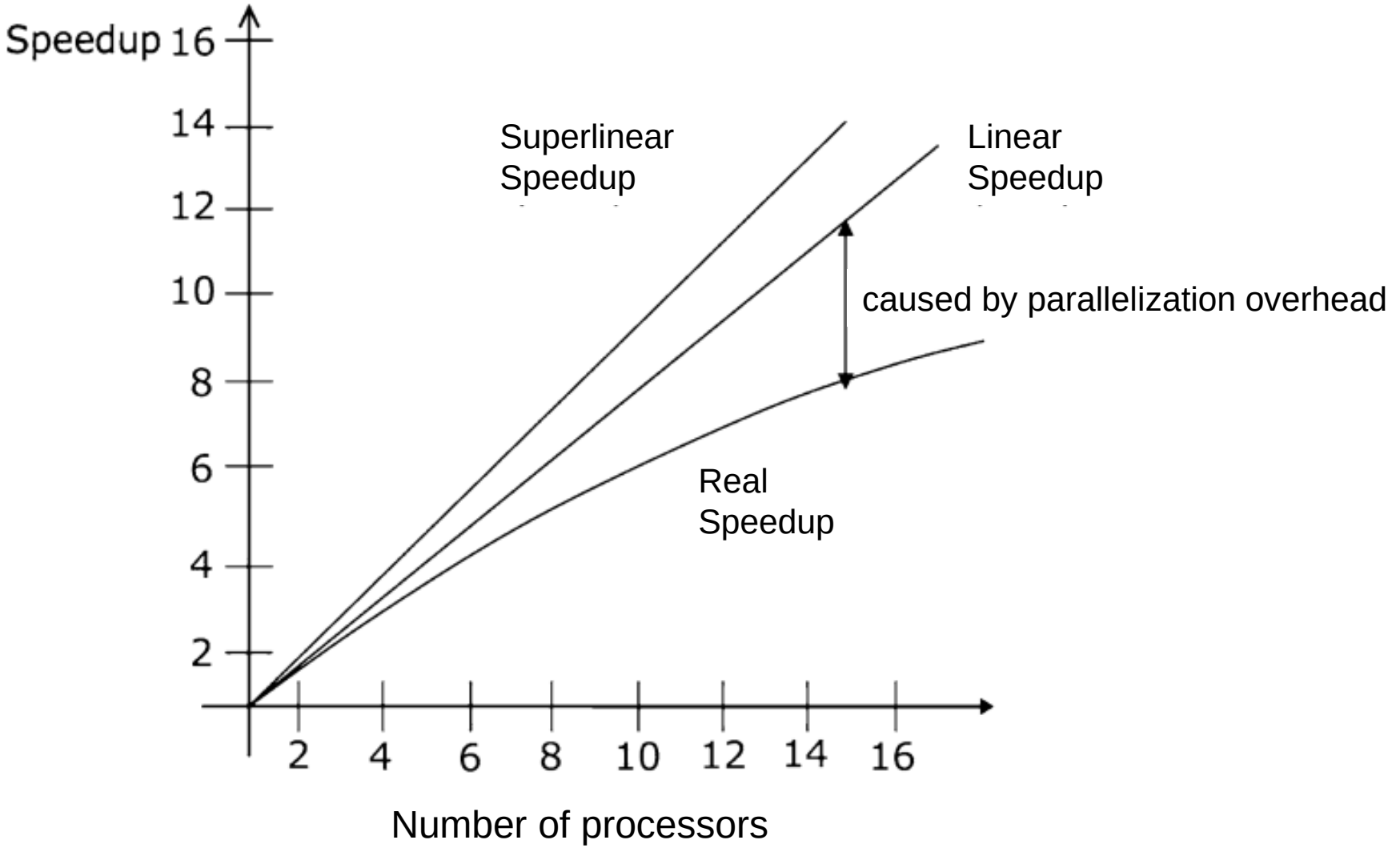
- It is the most important metric for quantifying the benefits of parallelization
- It is greater than 1 if the runtime of the parallel program is lower than the runtime of the non-parallel program

Speedup


Speedup
16
14
12
10
8
6
4
2
2
4
6
8
10
12
14
16
Superlinear Speedup
Linear Speedup
caused by parallelization overhead
Real Speedup
Number of processors

Discussion

- Why is the real speedup not a linear function?
- Is superlinear speedup possible?



Speedup

- Typically, the parallelization overhead increases if more processors are involved
- Consequently, we observe an increasing difference between the linear and real speedup
- Is superlinear speedup possible?
  - Hard question

Theoretical Bounds for the Speedup

- In practice, most programs cannot be fully parallelized
- There are sections which need be sequential, others can be executed in parallel
- Let f be the share of the program that needs to stay sequential
- Hence, $1 - f$ is the share of the program that can be parallelized
- The runtime of the sequential program can be calculated as:

$$T'(n) = f * T'(n) + (1 - f) * T'(n)$$

Amdahl‘s Law

- With parallelization, we cannot reduce the runtime of the sequential part $f * T'(n)$
- We split the execution of the parallelizable part to p processors
- Under the assumptions that all processors are equally fast and that the parallel program has the same of number of instructions as the sequential version, the following lower bound for the runtime of the parallel program exists:

$$T_p(n) \geq f * T'(n) + \frac{(1-f) * T'(n)}{p}$$

Amdahl‘s Law

- Inserting this inequation in the speedup formula gives us an upper bound for the speedup

$$S_p(n) \leq \frac{1}{f + \frac{(1-f)}{p}}$$

- This formula is known as *Amdahl’s law*
- Extreme values
  - f = 0: The full program can be parallelized. The maximum speedup is p.
  - f = 1: The program cannot be parallelized. The maximum speedup is 1.
  - p is infinity, f > 0: Even if we have infinite processors, the maximum speedup is 1/f.

Karp-Flatt Value

- If we rearrange Amdahl‘s law to the sequential share of the program f , we get:

$$f = \frac{1/S_p(n) - 1/p}{1 - 1/p}$$

- This formula is called the *Karp-Flatt value*
- In practice, we may not be able to determine the value of f from our code
- However, as we can easily measure program runtimes and hence speedups, we can get an empirical value of f using this formula

Amdahl‘s Law and Karp-Flatt Value

- What to learn from Amdahl‘s law and the Karp-Flatt value?

Massive parallelism (i.e., using many processors) only pays of if a large part of the program can be parallelized

Gustafson‘s Law

- Assumption of Amdahl‘s law: *the parallel program has the same of number of instructions as the sequential version*
- This also means that the problem size is kept constant
- Amdahl‘s law says: For that problem (input), you cannot be faster than 1/f-times the sequential program even if you have an infinite number of processors
- This might be true, but is a very pessimistic way of thinking parallelism

Gustafson‘s Law

- Gustafson found a more positive way of thinking about parallelism: *With a parallel program, we can solve larger problems in the same time as sequential programs*

Gustafson‘s Law

- Let a be the share of the runtime of the parallel program that is spent in the sequential part of the program
- Hence, the runtime of the parallel program can be expressed as:

$$T_p(n) = a * T_p(n) + (1 - a) * T_p(n)$$

- The runtime of a sequential program solving the same problem size would at most be:

$$T'(n) \leq a * T_p(n) + p * (1 - a) * T_p(n)$$

Gustafson‘s Law

- Inserting this relationship into the speedup formula gives us:

$$S_p(n) \leq a + p * (1 - a)$$

- This formula is called Gustafson‘s law
- It tells us: You cannot solve problems larger than a+p(1-a)-times the problem that the sequential program solves in the same runtime
- Extreme values
  - a = 0: The parallel program spends no time in the sequential part. We can solve p-times larger problems.
  - a = 1: We execute only the sequential part, speedup is 1

Amdahl‘s vs. Gustafson‘s Law

- Amdahl‘s Law limits the speed in which we can solve a given problem size
- Gustafson‘s Law limits the problem size which we can solve in a given time

Superlinear Speedups

- Both Amdahl‘s and Gustafson‘s law prohibit superlinear speedups
- However, for problems that can be fully parallelized, we can sometimes measure superlinear speedups
- Why?
  - The reasons for that are not related to parallelization
  - Often, they are related to memory or I/O effects, e.g., special parallel programs might require less updates to the main memory
- We might also achieve superlinear speedups if we compare with a bad sequential implementation ☺

Discussion

- Example: Consider a non-parallel program with a runtime of 100ms
- Consider different parallel versions of that program:
  - Version A has a runtime of 50ms and uses 2 processors
  - Version B has a runtime of 40ms and uses 5 processors
  - Version C has a runtime of 30ms and uses 20 processors
- Which one would you choose?

Parallelization Costs

- Although parallelization may reduce the runtime of a program, it comes with some costs as well
- A parallel program consumes more than one processor/core at a time, this resource is meanwhile not available to other programs
- The parallelization costs C_p take this into account and are defined as:

$$C_p = T_p(n) * p$$

- A parallel program is considered cost-optimal, if:

$$C_p = T'(n)$$

Parallelization Costs

- The parallelization costs correlate with the total time used on all processors/cores and therefore also with the total number of instructions executed
- Consider the example values from the discussion again

Program	Runtime	Number of processors	Speedup	Costs
Sequential	100	1	1	100
Parallel Verison A	50	2	2	100
Parallel Version B	40	5	2.5	200
Parallel Version C	30	20	3.33	600

Efficiency

- The efficiency $E_p(n)$ quantifies how efficient p processors are used for the parallel program

$$E_p(n) = \frac{S_p(n)}{p} = \frac{T'(n)}{p * T_p(n)} = \frac{T'(n)}{C_p(n)}$$

- If the efficiency is
  - Less than 1: The parallel program is sub-optimal in terms of its costs (often the case in reality)
  - Equal to 1: The parallel program is cost-optimal
  - Greater than 1: We have a superlinear speedup. The parallel program needs less operations in total than the sequential program

Efficiency

- Consider the example values from the discussion again

Program	Runtime	Number of processors	Speedup	Costs	Efficiency
Sequential	100	1	1	100	1
Parallel Verison A	50	2	2	100	1
Parallel Version B	40	5	2.5	200	0.5
Parallel Version C	30	20	3.33	600	0.17

How to make a program parallel?

Inherent Parallelism

- A problem has inherent parallelism if it can be decomposed into many independent tasks with minimal communication
- Also called *embarrassingly parallel problems*
- Examples
  - Image filtering
    - Task: Find all images from a set showing a specific person
    - Each image can be processed in parallel
  - Word counting
    - Task: Get the number of words in one or multiple files
    - Each file can be processed in parallel
  - Hacking
    - Task: Search for passwords/PINs
    - Every possible password/PIN can be evaluated independently

Discussion

- Can you name other inherent parallel problems?
- Can you give counterexamples (inherent sequential problems)?

- Some inherent sequential problems include
  - Serial database transactions with strong consistency constraints
  - Calculation of Fibonacci numbers
  - Linked list traversal

Group Activity

- I will hand out some papers with numbers on it
- You will get a task and will measure the time you need to complete the task
- Your task: Calculate the sum of all numbers that I handed out to you

Divide and Conquer

- *Divide and conquer* is the most famous general approach to explore parallelism for a problem

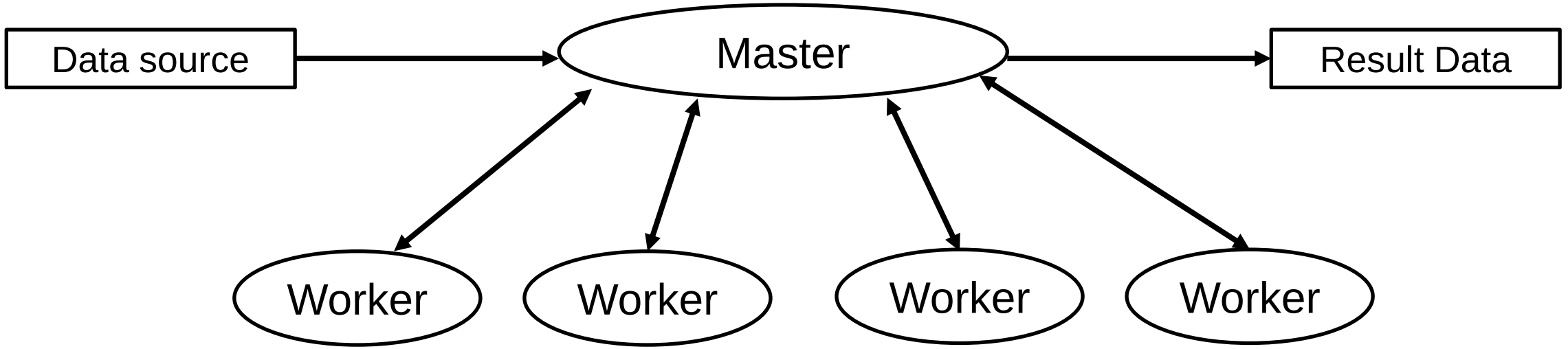
```
procedure Divide_and_Conquer (problem)
begin
        if base_case then
                solve problem
        else
                partition problem into subproblems L and R;
                Divide_and_Conquer(L);
                Divide_and_Conquer(R);
                combine solutions to problems L and R;
        end if;
end;
```

Divide and Conquer

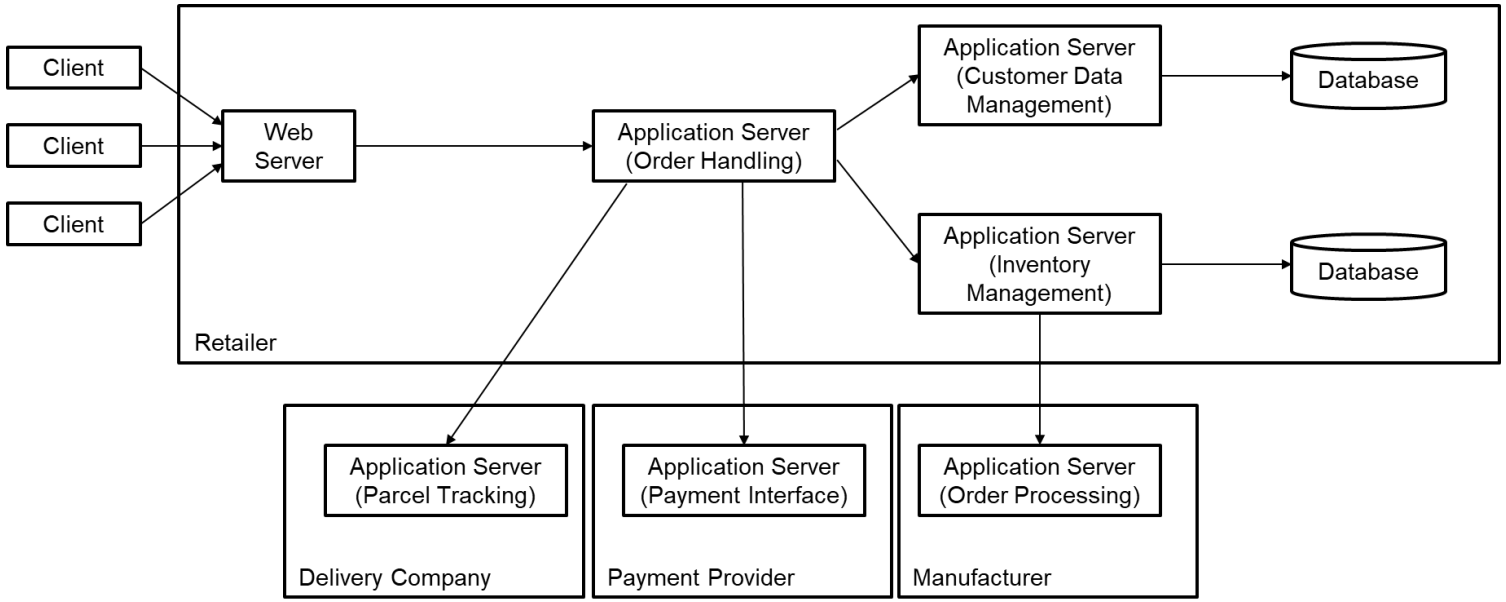
- Example from “real life”: Find the tallest person in a large crowd of people
  - Divide the crowd into several smaller groups
  - Find simultaneously the tallest person in every group
  - Compare the tallest persons from every group only
- Analogy from programming: Find the maximum in a large array
  - Split the array to a left and right half
  - Find maximum of left and right half in parallel
  - Compare left and right maximum to get global maximum

Data Parallelism

- Also called domain decomposition
- Idea: The same operation is applied simultaneously to different pieces of data
- Example: Increment all elements in an array
- Common scheme: Master and worker



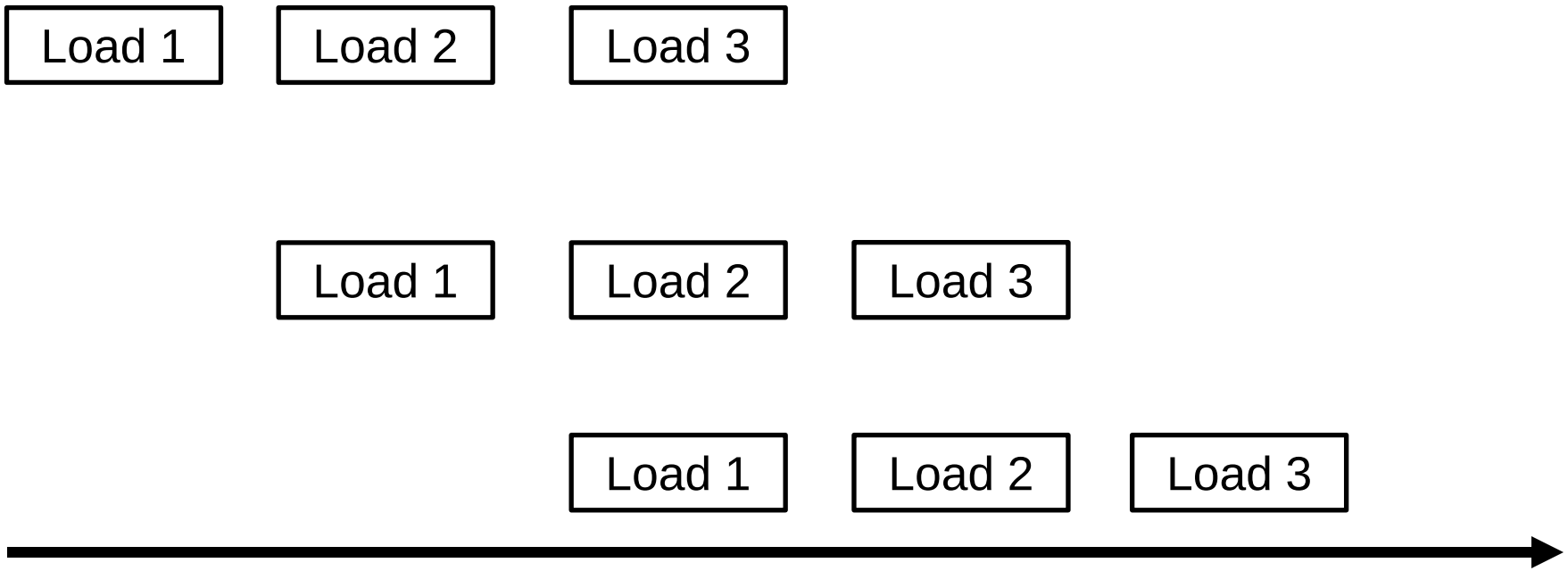
Task Parallelism



- Also called function decomposition
- Idea: Different functions (tasks) of a program run in parallel on the same or different data
- Example: Web services
  - Every web service has a specific task (e.g., validating customer orders, sending out confirmation mails, updating the inventory)
  - Every web service executes this task on different data (e.g., orders)

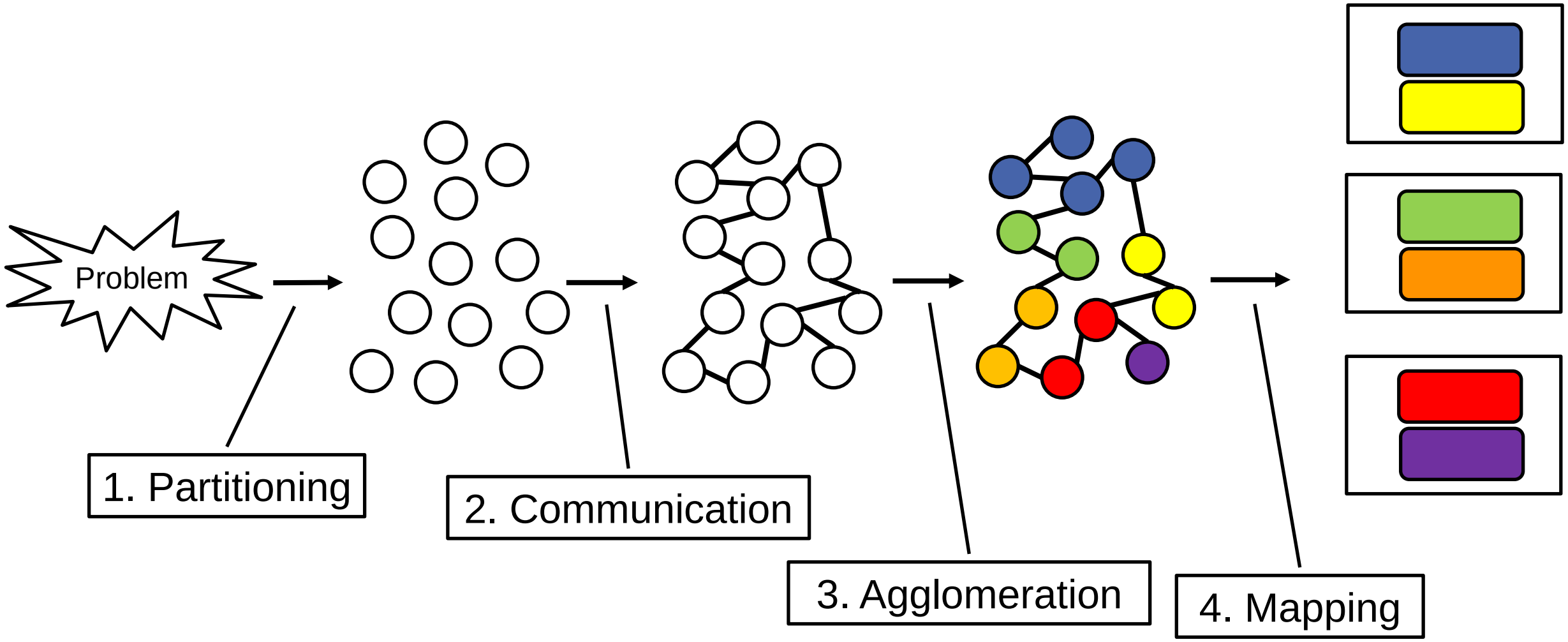
Pipeline Parallelism

- Special case of function decomposition where tasks are arranged in stages, and data flows through them like an assembly line
- Example: Laundry pipeline



Designing Parallel Programs

- Also called Foster's Methodology



Foster‘s Methodology

- Partitioning
  - Divide the problem into smaller pieces of work
  - Identify tasks or data that can be executed independently
- Communication
  - Identify what data is shared, when communication is required, how much data is exchanged
  - Minimize communication where possible
- Agglomeration
  - Group tasks to reduce overhead and improve efficiency
  - Reduce communication frequency
- Mapping
  - Assign tasks to processors